COMP102: Data Structures with Python

Mentoring Worksheet — BSTs and AVL Trees

University Mohammed VI Polytechnic
School of Applied Science and Engineering (SASE)

Minimal node class

All problems use this basic node structure unless a problem gives another one.

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right
```

Exercise 1: Build and read a BST

The goal is to implement insertion, then use inorder traversal to verify the BST structure.

Paper trace first

Starting from an empty tree, insert the values in this order:

$$8,\ 3,\ 10,\ 1,\ 6,\ 14,\ 4,\ 7$$

Draw the final BST. Then write its inorder traversal.

Coding task

Complete the functions below.

```
class Node:
    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right


def insert(root, x):
    """Insert x into the BST and return the root."""
    # TODO: empty tree case

    # TODO: go left if x is smaller

    # TODO: go right if x is larger

    # TODO: ignore duplicates

    return root


def build_bst(values):
    """Build a BST by inserting values from left to right."""
    root = None
    # TODO: insert every value into root
    return root


def inorder(root):
    """Return the inorder traversal as a list."""
    # TODO
    pass
```

Expected tests

Your code should satisfy:

```
root = build_bst([8, 3, 10, 1, 6, 14, 4, 7])
print(inorder(root))
# [1, 3, 4, 6, 7, 8, 10, 14]

root = build_bst([5, 5, 3, 7, 3])
print(inorder(root))
# [3, 5, 7]
```

Common bug

A frequent mistake is to create a new node in the correct place but forget to reconnect it:

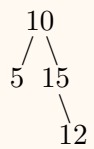
```
insert(root.left, x)        # wrong if the return value is ignored
root.left = insert(root.left, x)    # correct
```

Exercise 2: Validate a BST with bounds

A BST is not validated by checking only direct children. The whole left subtree must be smaller than the current node, and the whole right subtree must be larger.

Counterexample

This tree is **not** a valid BST:



The local comparison `12 > 5` is true, but `12` is still invalid because it is inside the left subtree of `10`.

Coding task

Complete `is_bst(root)` using lower and upper bounds.

```
def is_bst(root):
    """Return True if root is a valid strict BST."""
    def check(node, low, high):
        # TODO: empty tree is valid

        # TODO: node.value must be strictly between low and high

        # TODO: recursively check left and right subtrees
        # Left subtree upper bound becomes node.value.
        # Right subtree lower bound becomes node.value.
        pass

    return check(root, None, None)
```

Expected tests

Your code should satisfy:

```
valid = Node(8, Node(3, Node(1), Node(6)), Node(10, None, Node(14)))
print(is_bst(valid))
# True

invalid = Node(10, Node(5, None, Node(12)), Node(15))
print(is_bst(invalid))
# False

duplicate = Node(5, Node(3), Node(5))
print(is_bst(duplicate))
# False
```

Common bug

This style is insufficient:

```
if root.left and root.left.value >= root.value:
    return False
if root.right and root.right.value <= root.value:
    return False
```

It only checks direct children. It misses violations deeper inside a subtree.

Exercise 3: Repair an AVL insertion

An AVL tree is a BST with an extra balance rule:

$$|\,\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1.$$

For this exercise, use:

$$\text{height}(\text{None}) = 0, \qquad \text{height}(\text{leaf}) = 1.$$

Rotation cases

For a node `z` that becomes unbalanced after insertion:

- Left-left case: rotate right at `z`.
- Right-right case: rotate left at `z`.
- Left-right case: rotate left at `z.left`, then rotate right at `z`.
- Right-left case: rotate right at `z.right`, then rotate left at `z`.

Paper trace first

For each insertion sequence, write the imbalance case and the final root after rebalancing:

(1) `30, 20, 10`

(2) `10, 20, 30`

(3) `30, 10, 20`

(4) `10, 30, 20`

Coding task

Complete the missing parts of AVL insertion. The rotation functions are given.

```
class AVLNode:
    def __init__(self, value, left=None, right=None, height=1):
        self.value = value
        self.left = left
        self.right = right
        self.height = height

def height(node):
    return 0 if node is None else node.height

def update_height(node):
    if node is not None:
        node.height = 1 + max(height(node.left), height(node.right))

def balance_factor(node):
    return 0 if node is None else height(node.left) - height(node.right)

def right_rotate(y):
    x = y.left
    t2 = x.right
    x.right = y
    y.left = t2
    update_height(y)
    update_height(x)
    return x

def left_rotate(x):
    y = x.right
    t2 = y.left
    y.left = x
    x.right = t2
    update_height(x)
    update_height(y)
    return y

def avl_insert(root, x):
    # Step 1: normal BST insertion
    if root is None:
        return AVLNode(x)
    if x < root.value:
        root.left = avl_insert(root.left, x)
    elif x > root.value:
        root.right = avl_insert(root.right, x)
    else:
        return root

    # Step 2: update height
    # TODO

    # Step 3: compute balance
    # TODO

    # Step 4: repair the four cases
    # Left-left
    if balance > 1 and x < root.left.value:
        # TODO
        pass

    # Right-right
    if balance < -1 and x > root.right.value:
        # TODO
        pass

    # Left-right
    if balance > 1 and x > root.left.value:
        # TODO: rotate left on root.left, then right on root
        pass

    # Right-left
    if balance < -1 and x < root.right.value:
        # TODO: rotate right on root.right, then left on root
        pass

    return root
```

Expected tests

Your code should satisfy:

```
def inorder(root):
    if root is None:
        return []
    return inorder(root.left) + [root.value] + inorder(root.right)

root = None
for x in [30, 20, 10]:
    root = avl_insert(root, x)
print(root.value, inorder(root))
# 20 [10, 20, 30]

root = None
for x in [30, 10, 20]:
    root = avl_insert(root, x)
print(root.value, inorder(root))
# 20 [10, 20, 30]
```

Common bug

Students often rotate correctly but forget to update heights in the correct order. After a rotation, update the lower node first, then the new root.

Check

Write one sentence answering each question:

(1) Why does inorder traversal of a BST return sorted values?

(2) Why is validating a BST with only direct-child comparisons wrong?

(3) What problem do AVL rotations solve?